Asigna Nombres Significativos a Tus Fotos con IA



Introducción

Las imágenes, ricas en información no aprovechada, a menudo quedan fuera del radar de los motores de búsqueda y sistemas de datos. Transformar estos datos visuales en un lenguaje legible por máquinas no es una tarea fácil, pero es aquí donde la IA de subtitulación de imágenes resulta útil. Aquí te mostramos cómo la IA de subtitulación de imágenes puede marcar la diferencia:

- Mejora la accesibilidad: Ayuda a las personas con discapacidad visual a comprender el contenido visual.
- Mejora el SEO: Asiste a los motores de búsqueda en la identificación del contenido de las imágenes.
- Facilita el descubrimiento de contenido: Permite un análisis y categorización eficientes de grandes bases de datos de imágenes.
- Apoya las redes sociales y la publicidad: Automatiza la generación de descripciones atractivas para contenido visual.
- Aumenta la seguridad: Proporciona descripciones en tiempo real de las actividades en grabaciones de video.
- Ayuda en la educación y la investigación: Asiste en la comprensión e interpretación de materiales visuales.
- Ofrece soporte multilingüe: Genera subtítulos de imágenes en varios idiomas para audiencias internacionales.
- Permite la organización de datos: Ayuda a gestionar y categorizar grandes conjuntos de datos visuales.
- Ahorra tiempo: La subtitulación automatizada es más eficiente que los esfuerzos manuales.

The content of this lab is licensed under Apache 2.0

- Aumenta la participación del usuario: Los subtítulos detallados pueden hacer que el contenido visual sea más atractivo e informativo.

Objetivos de aprendizaje

Al final de este proyecto, podrás:

- Implementar una herramienta de subtitulación de imágenes utilizando el modelo BLIP de los Transformers de Hugging Face.
- Usar Gradio para proporcionar una interfaz amigable para tu aplicación de subtitulación de imágenes.
- Adaptar la herramienta a escenarios comerciales del mundo real, demostrando sus aplicaciones prácticas.



Generado por IA y mejorado por humanos

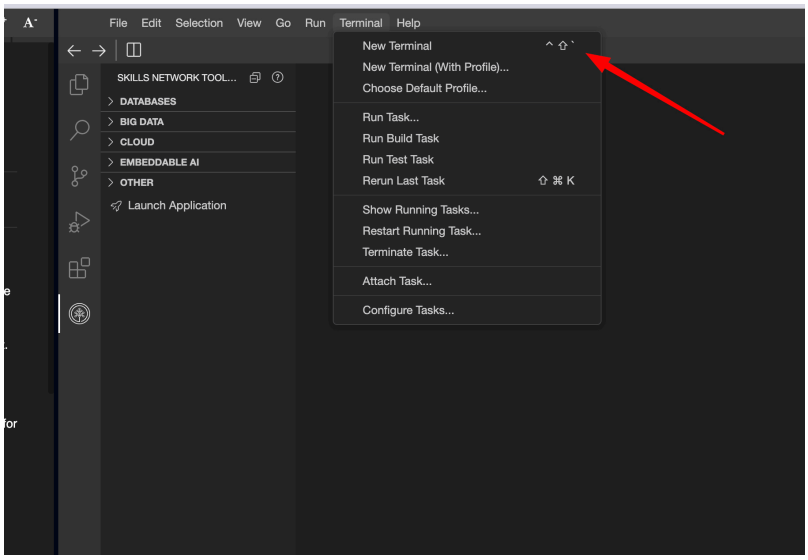
The content of this lab is licensed under Apache 2.0

Configuración del entorno e instalación de bibliotecas

En este proyecto, para construir una aplicación de IA, utilizarás la interfaz Gradio proporcionada por Hugging Face.

Vamos a configurar el entorno y las dependencias para este proyecto. Abre una nueva terminal y asegúrate de estar en el directorio `home/project`.

Abre una nueva terminal:



Abre una nueva terminal

Crea un entorno virtual de Python e instala Gradio utilizando los siguientes comandos en la terminal:

```bash
pip3 install virtualenv
virtualenv my_env # create a virtual environment my_env
source my_env/bin/activate # activate my_env
```

Run

Luego, instala las bibliotecas necesarias en el entorno:

The content of this lab is licensed under Apache 2.0

bash

```
# installing required libraries in my_env
pip install langchain==0.1.11 gradio==5.23.2 transformers==4.38.2 bs4==0.0
```

Run

Tómate una taza de café, tomará 5 minutos.

plaintext

```
     )  (
    (   ) )
     ) ( (
   _______)_
 .-'---------|
( C|/\/\/\/\/|
 '-./\/\/\/\/|
   '_________'
    '-------'
```

Ahora, tu entorno está listo para crear archivos de Python.

The content of this lab is licensed under Apache 2.0

Generación de descripciones de imágenes con el modelo BLIP

Introducción: Hugging Face, Transformers y BLIP

Hugging Face es una organización que se centra en el procesamiento de lenguaje natural (NLP) y la inteligencia artificial (IA). La organización es conocida por su biblioteca de código abierto llamada "***Transformers***" que proporciona miles de modelos preentrenados a la comunidad. La biblioteca admite una amplia gama de tareas de NLP, como traducción, resumido, generación de texto y más. ***Transformers*** ha contribuido significativamente a los recientes avances en NLP, ya que ha hecho que modelos de vanguardia, como BERT, GPT-2 y GPT-3, sean accesibles para investigadores y desarrolladores en todo el mundo.

La biblioteca Transformers incluye un modelo que se puede utilizar para capturar información de imágenes. El modelo BLIP, o Bootstrapping Language-Image Pre-training, es una herramienta que ayuda a las computadoras a entender y generar lenguaje basado en imágenes. Es como enseñar a una computadora a mirar una imagen y describirla, o responder preguntas sobre ella.

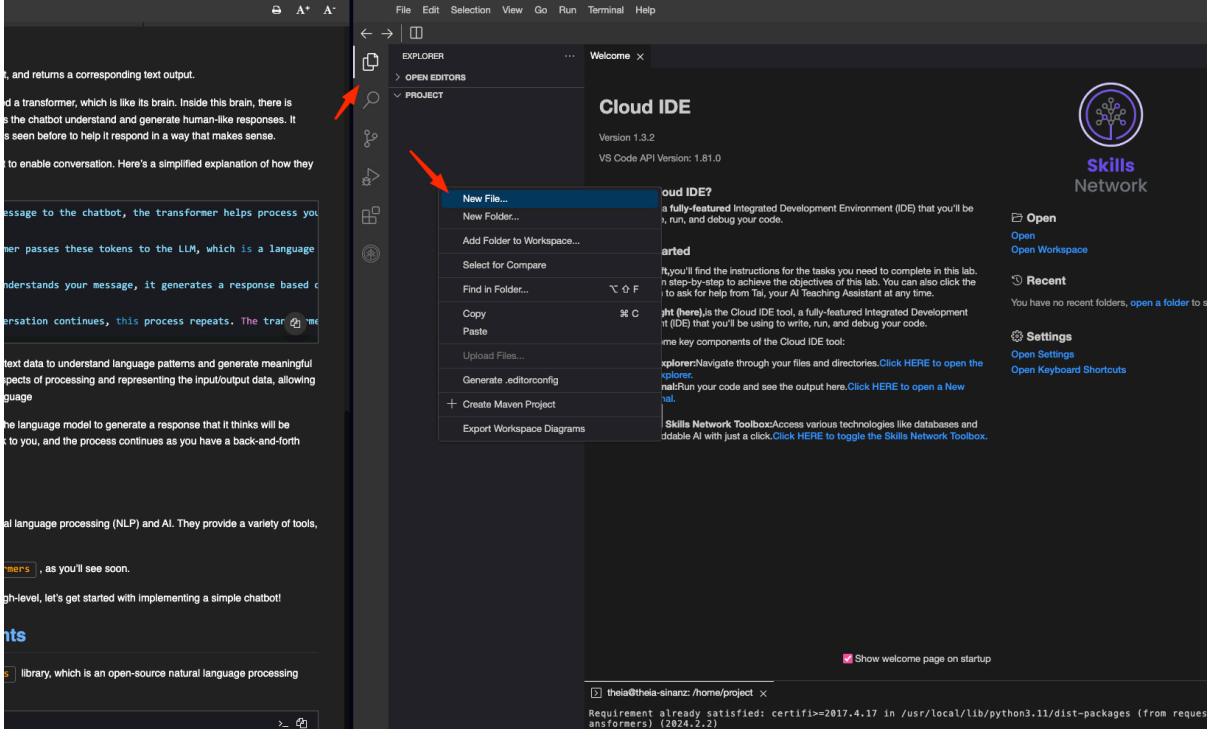
¡Bien, ahora que sabes lo que BLIP puede hacer, comencemos a implementar una aplicación de IA para la generación de descripciones de imágenes!

Paso 1: Importa las herramientas necesarias de la biblioteca transformers

Ya has instalado el paquete `transformers` durante la configuración del entorno.

En el directorio `project`, crea un archivo de Python. Haz clic en `Explorador de Archivos`, luego haz clic derecho en el área del explorador y selecciona `Nuevo Archivo`. Nombra este nuevo archivo `image_cap.py`. Copia los diversos segmentos de código a continuación y pégalos en el archivo de Python.

The content of this lab is licensed under Apache 2.0



Nuevo archivo

Utilizarás `AutoProcessor` y `BlipForConditionalGeneration` de la biblioteca `transformers`.

"AutoProcessor" y "BlipForConditionalGeneration" son componentes del modelo BLIP, que es un modelo de visión-lenguaje disponible en la biblioteca Transformers de Hugging Face.

- **AutoProcessor** : Esta es una clase de procesador que se utiliza para el preprocesamiento de datos para el modelo BLIP. Envuelve un procesador de imágenes BLIP y un tokenizador OPT/T5 en un solo procesador. Esto significa que puede manejar tanto datos de imagen como de texto, preparándolos para la entrada en el modelo BLIP.

> *Nota: Un tokenizador es una herramienta en el procesamiento de lenguaje natural que descompone el texto en unidades más pequeñas y manejables (tokens), como palabras o frases, lo que permite a los modelos analizar y comprender el texto.*

- **BlipForConditionalGeneration** : Esta es una clase de modelo que se utiliza para la generación de texto condicional dado una imagen y un texto opcional. En otras palabras, puede generar texto basado en una imagen de entrada y una pieza de texto opcional. Esto lo hace útil para tareas como la generación de descripciones de imágenes o la respuesta a preguntas visuales, donde el

The content of this lab is licensed under Apache 2.0

modelo necesita generar texto que describa una imagen o responda a una pregunta sobre una imagen.

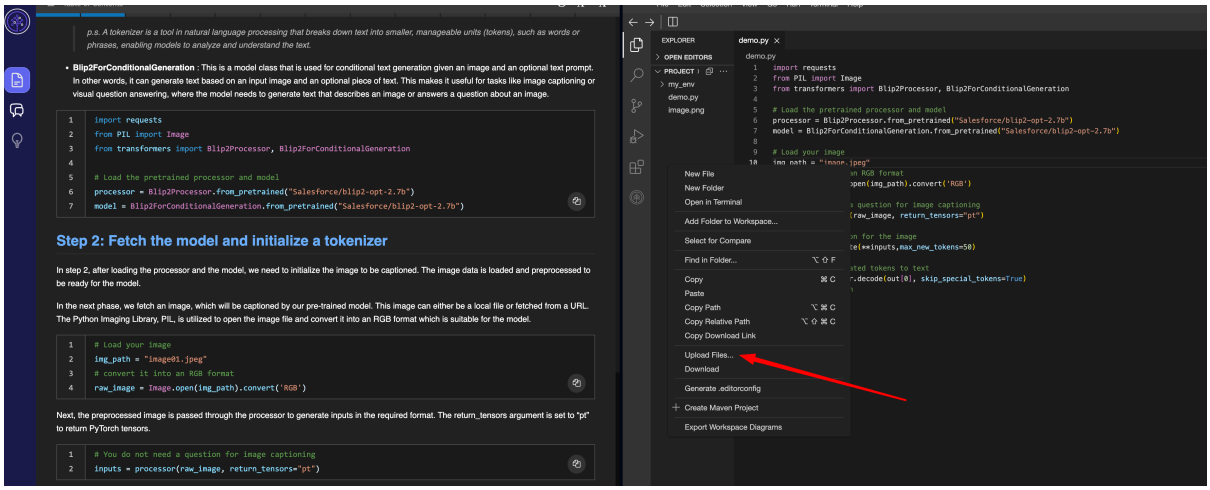
```python
import requests
from PIL import Image
from transformers import AutoProcessor, BlipForConditionalGeneration

# Load the pretrained processor and model
processor = AutoProcessor.from_pretrained("Salesforce/blip-image-captionin
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-imag
```

Paso 2: Cargar y Preprocesar una Imagen

Después de cargar el procesador y el modelo, necesitas inicializar la imagen que será subtitulada. Los datos de la imagen deben ser cargados y preprocesados para estar listos para el modelo.

Para cargar la imagen, haz clic derecho en cualquier parte del Explorador (en el lado izquierdo del panel de código) y selecciona `Upload Files...` (como se muestra en la imagen a continuación). Puedes subir cualquier imagen desde tus archivos locales y modificar el `img_path` de acuerdo con el nombre de la imagen.



Subir archivos

En la siguiente fase, obtienes una imagen, que será subtitulada por tu modelo preentrenado. Esta imagen puede ser un archivo local o recuperada de una URL. La Biblioteca de Imágenes de Python, PIL, se utiliza para abrir el archivo de imagen y

The content of this lab is licensed under Apache 2.0

```python
# Load your image, DON'T FORGET TO WRITE YOUR IMAGE NAME
img_path = "YOUR IMAGE NAME.jpeg"
# convert it into an RGB format
image = Image.open(img_path).convert('RGB')
```

A continuación, la imagen preprocesada se pasa por el procesador para generar entradas en el formato requerido. El argumento return_tensors se establece en "pt" para devolver tensores de PyTorch.

```python
# You do not need a question for image captioning
text = "the image of"
inputs = processor(images=image, text=text, return_tensors="pt")
```

Luego, pasas estas entradas al método generate de tu modelo. El argumento `max_length=50` especifica que el modelo debe generar un título de hasta 50 tokens de longitud.

> *Los dos asteriscos (**) en Python se utilizan en las llamadas a funciones para desempaquetar diccionarios y pasar elementos del diccionario como argumentos de palabra clave a la función.* `**inputs` *está desempaquetando el diccionario de entradas y pasando sus elementos como argumentos al modelo.*

```python
# Generate a caption for the image
outputs = model.generate(**inputs, max_length=50)
```

Finalmente, la salida generada es una secuencia de tokens. Para transformar estos tokens en texto legible por humanos, utilizas el método decode proporcionado por el procesador. El argumento `skip_special_tokens` se establece en `True` para ignorar los tokens especiales en el texto de salida.

The content of this lab is licensed under Apache 2.0

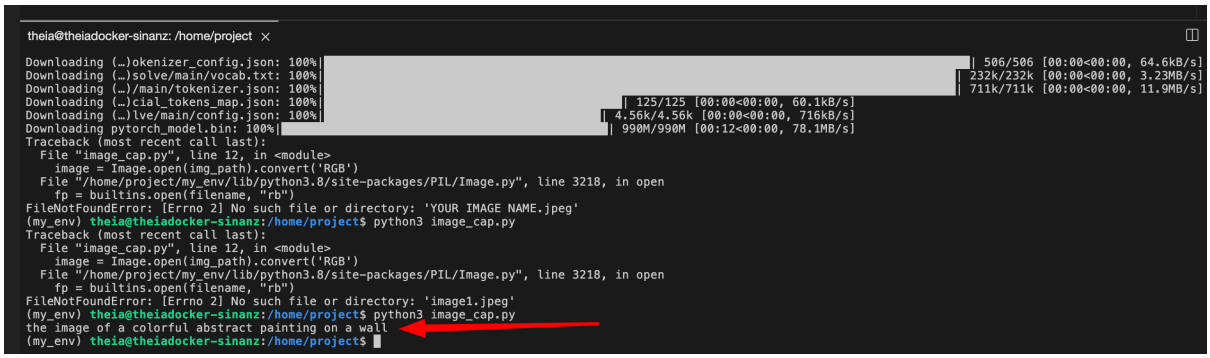
```python
# Decode the generated tokens to text
caption = processor.decode(outputs[0], skip_special_tokens=True)
# Print the caption
print(caption)
```

Guarda tu archivo Python y ejecútalo para ver el resultado.

```bash
python3 image_cap.py
```

Run

Y tienes el pie de foto de la imagen, generado por tu modelo. Este pie de foto es una representación textual del contenido de la imagen, según la interpretación del modelo BLIP.



Salida del pie de foto

The content of this lab is licensed under Apache 2.0

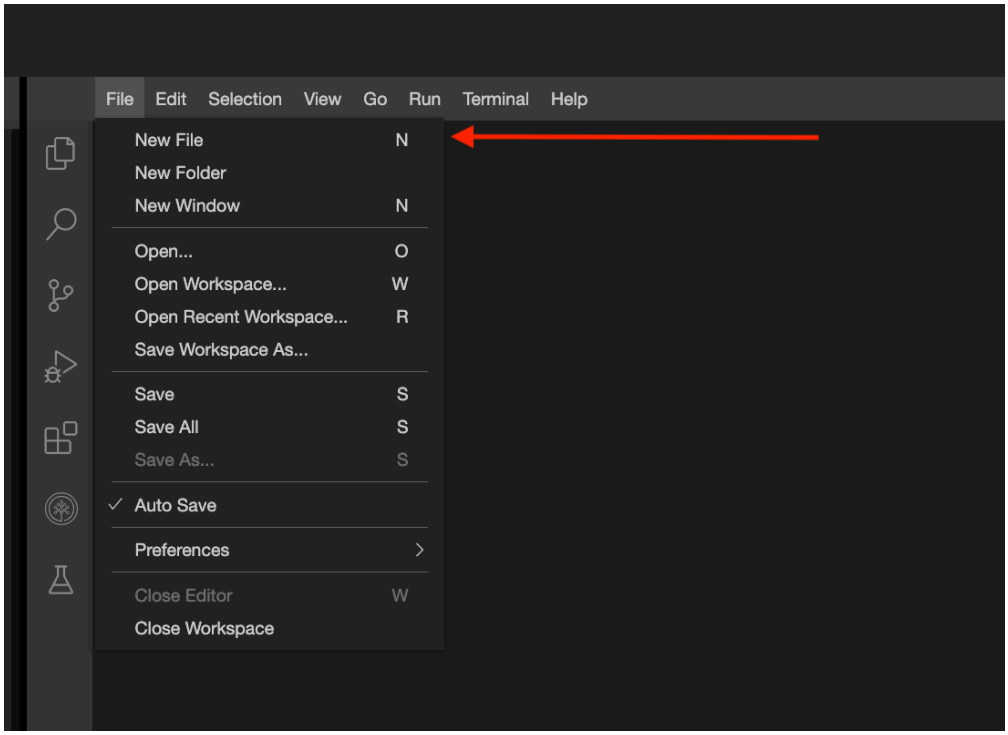
Aplicación de subtitulado de imágenes con Gradio

Ahora que entiendes el mecanismo de subtitulado de imágenes, vamos a crear una aplicación adecuada con una interfaz intuitiva. Puedes utilizar Gradio, una herramienta proporcionada por Hugging Face, para este propósito. Para comenzar, tendrás una breve introducción a Gradio. Después de eso, como ejercicio, se te asignará la tarea de implementar la aplicación de subtitulado de imágenes utilizando la interfaz de Gradio.

Inicio rápido con Gradio: Creando una demostración simple

Familiaricémonos con Gradio creando una aplicación simple:

Aún en el directorio `project`, crea un archivo Python y llámalo `hello.py`.



Nuevo archivo

Abre `hello.py`, copia y pega el siguiente código Python y guarda el archivo.

The content of this lab is licensed under Apache 2.0

```python
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

demo = gr.Interface(fn=greet, inputs="text", outputs="text")

demo.launch(server_name="0.0.0.0", server_port= 7860)
```

El código crea una interfaz de Gradio llamada `demo` utilizando la clase *gr.Interface*. Envuelve la función `greet` con una interfaz de usuario simple de texto a texto con la que puedes interactuar.

La clase **gr.Interface** se inicializa con 3 parámetros requeridos:

- fn: la función a la que se le aplicará una interfaz
- inputs: qué componente(s) utilizar para la entrada (por ejemplo, "texto", "imagen" o "audio")
- outputs: qué componente(s) utilizar para la salida (por ejemplo, "texto", "imagen" o "etiqueta")

La última línea `demo.launch()` lanza un servidor para servir tu `demo`.

Lanzando la aplicación demo

Ahora vuelve a la terminal y asegúrate de que el nombre del entorno virtual `my_env` se muestre al principio de la línea.

Ahora ejecuta el siguiente comando para ejecutar el script de Python.

```bash
python3 hello.py
```

Run

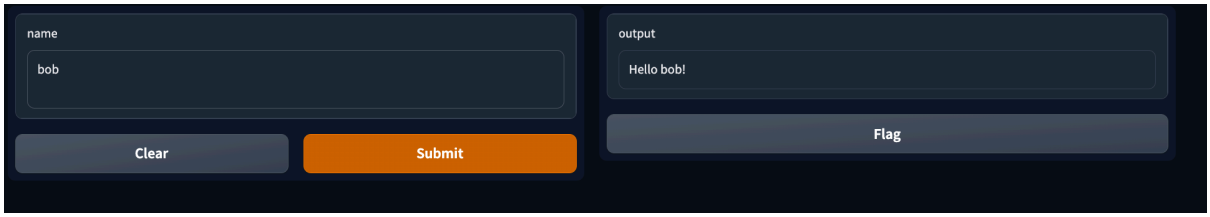
Como el código de Python se sirve desde el host local, haz clic en el botón de abajo y podrás ver la aplicación simple que creaste. ¡Siéntete libre de experimentar con la entrada y salida de la aplicación web!

The content of this lab is licensed under Apache 2.0

Haz clic aquí para ver la aplicación:

Aplicación Web

Deberías ver lo siguiente, aquí el nombre ingresado es bob:



Entrada y salida

Si terminas de jugar con la aplicación y quieres salir, **presiona `ctrl+c` en la terminal y cierra la pestaña de la aplicación**.

Acabas de tener un primer vistazo de la interfaz de Gradio, ¿es fácil, verdad? Si deseas aprender un poco más sobre la personalización en Gradio, estás invitado a realizar el proyecto guiado llamado **Dale vida a tu modelo de Machine Learning con Gradio**. Puedes encontrarlo en **Cursos y Proyectos** en cognitiveclass.ai!

The content of this lab is licensed under Apache 2.0

Ejercicio: Implementar una aplicación de subtitulado de imágenes con Gradio

En este ejercicio, seguirás los pasos para crear una aplicación web que genera subtítulos para imágenes utilizando el modelo BLIP y la biblioteca Gradio. Sigue los pasos a continuación:

Paso 1: Configurar el entorno

- Asegúrate de tener las bibliotecas necesarias instaladas. Ejecuta `pip install gradio transformers Pillow` para instalar Gradio, Transformers y Pillow.
- Importa las bibliotecas requeridas:

Ahora, vamos a crear un nuevo archivo Python y llamarlo `image_captioning_app.py`.

```python
import gradio as gr
import numpy as np
from PIL import Image
from transformers import AutoProcessor, BlipForConditionalGeneration
```

Paso 2: Cargar el modelo preentrenado

- Cargar el procesador y el modelo preentrenados:

```python
processor = # write your code here
model = # write your code here
```

Paso 3: Definir la función de subtitulado de imágenes

- Define la función `caption_image` que toma una imagen de entrada y devuelve

The content of this lab is licensed under Apache 2.0

```python
def caption_image(input_image: np.ndarray):
    # Convert numpy array to PIL Image and convert to RGB
    raw_image = Image.fromarray(input_image).convert('RGB')

    # Process the image


    # Generate a caption for the image


    # Decode the generated tokens to text and store it into `caption`


    return caption
```

Paso 4: Crear la interfaz de Gradio

- Usa la clase `gr.Interface` para crear la interfaz de la aplicación web:

```python
iface = gr.Interface(
    fn=caption_image,
    inputs=gr.Image(),
    outputs="text",
    title="Image Captioning",
    description="This is a simple web app for generating captions for imag
)
```

Paso 5: Iniciar la Aplicación Web

- Inicia la aplicación web llamando al método `launch()`:

```python
iface.launch()
```

▶ Haz clic aquí para la respuesta

The content of this lab is licensed under Apache 2.0

Paso 6: Ejecuta la aplicación

- Guarda el código completo en un archivo Python, por ejemplo, `image_captioning_app.py`.
- Abre una terminal o símbolo del sistema, navega al directorio donde se encuentra el archivo y ejecuta el comando

```bash
python3 image_captioning_app.py
```

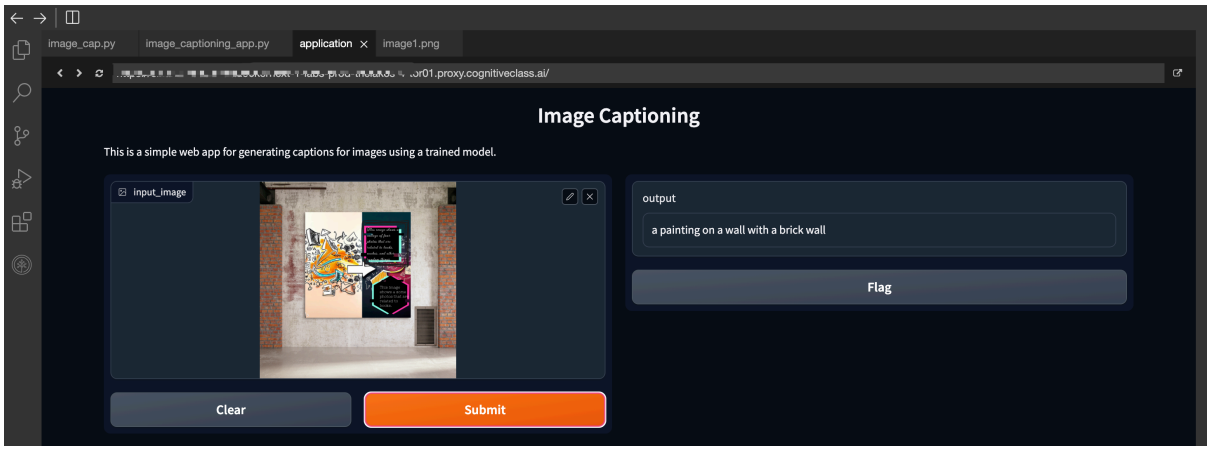
Run

Haz clic aquí para iniciar tu aplicación web:

Aplicación Web

Presiona `ctrl + c` para salir de la aplicación.

Tendrás una salida como esta en la nueva ventana:



Si estás ejecutando localmente: Interactúa con la aplicación web:

- La aplicación web debería comenzar a ejecutarse y mostrar una URL donde puedes acceder a la interfaz.
- Abre la URL proporcionada en un navegador web (en la terminal).
- Deberías ver una interfaz con un cuadro para subir imágenes.

¡Felicidades! Has creado una aplicación web de generación de descripciones de imágenes usando Gradio y el modelo BLIP. Puedes personalizar aún más la interfaz, modificar el código o experimentar con diferentes modelos y configuraciones para

The content of this lab is licensed under Apache 2.0

The content of this lab is licensed under Apache 2.0

Escenario: Cómo el subtitulado de imágenes ayuda a un negocio

Escenario empresarial en noticias y medios:

Una agencia de noticias publica cientos de artículos diariamente en su sitio web. Cada artículo contiene varias imágenes relevantes para la historia. Escribir subtítulos apropiados y descriptivos para cada imagen manualmente es una tarea tediosa y puede ralentizar el proceso de publicación.

En este escenario, tu programa de subtitulado de imágenes puede acelerar el proceso:

1. Los periodistas escriben sus artículos y seleccionan imágenes relevantes para acompañar la historia.

2. Estas imágenes se introducen en el programa de subtitulado de imágenes (en lugar de insertar manualmente la descripción para cada imagen).

3. El programa procesa estas imágenes y genera un archivo de texto con los subtítulos sugeridos para cada imagen.

4. Los periodistas o editores revisan estos subtítulos. Pueden usarlos tal como están, o pueden modificarlos para que se ajusten mejor al contexto del artículo.

5. Estos subtítulos aprobados sirven entonces para dos propósitos:

    - Accesibilidad mejorada: Los subtítulos se integran como texto alternativo (texto alt) para las imágenes en el artículo en línea. Los usuarios con discapacidad visual, que utilizan lectores de pantalla, pueden entender el contexto de las imágenes a través de estas descripciones. Esto les ayuda a tener una experiencia de consumo de contenido similar a la de los usuarios videntes, adhiriéndose a los principios de diseño inclusivo y accesible.

    - Mejora del SEO: Las imágenes correctamente subtituladas con palabras clave relevantes mejoran el SEO del artículo. Los motores de búsqueda como Google consideran el texto alt al indexar, y esto ayuda a que el

The content of this lab is licensed under Apache 2.0

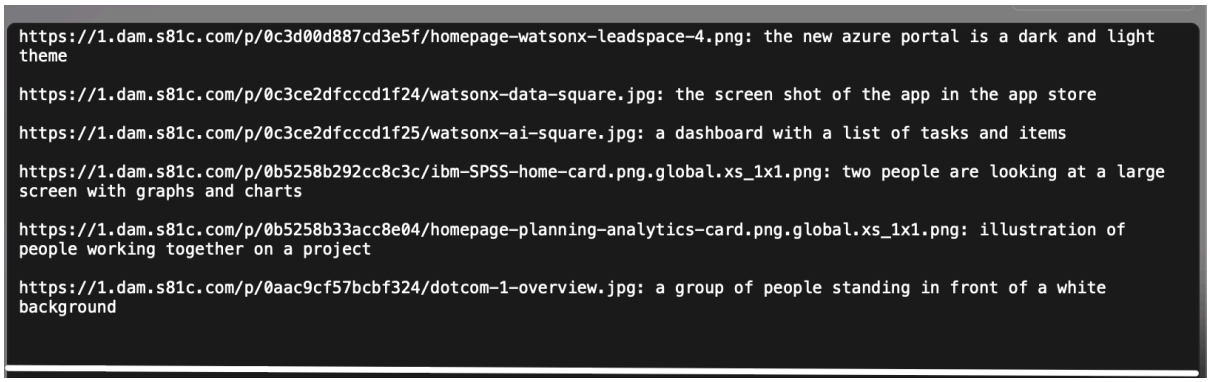
el tráfico orgánico al sitio web de la agencia. Esto es especialmente útil para los resultados de búsqueda de imágenes.

6. Una vez que los subtítulos son aprobados, se añaden a las imágenes en el artículo en línea.

Al integrar este proceso, la agencia no solo acelera su proceso de publicación, sino que también asegura que todas las imágenes vengan con descripciones apropiadas, mejorando la accesibilidad para los lectores con discapacidad visual y mejorando el SEO del sitio web. De esta manera, la agencia amplía su alcance y compromiso con una base de audiencia más diversa.

Implementemos una herramienta de subtitulado de imágenes automatizada

En esta sección, implementas un programa de subtitulado de imágenes automatizado que funciona directamente desde una URL. El usuario proporciona la URL, y el código genera subtítulos para las imágenes encontradas en la página web. La salida es un archivo de texto que incluye todas las URLs de las imágenes junto con sus respectivos subtítulos (como la imagen a continuación). Para lograr esto, utilizas BeautifulSoup para analizar el contenido HTML de la página y extraer las URLs de las imágenes.



URLs de imágenes

Comencemos:

Primero, envías una solicitud HTTP a la URL proporcionada y recuperas el contenido de la página web. Este contenido es luego analizado por BeautifulSoup, que crea un

The content of this lab is licensed under Apache 2.0

árbol de análisis, ya que contienen los enlaces a las imágenes alojadas en la página web.

```python
# URL of the page to scrape
url = "https://en.wikipedia.org/wiki/IBM"

# Download the page
response = requests.get(url)

# Parse the page with BeautifulSoup
soup = BeautifulSoup(response.text, 'html.parser')
```

Después de extraer estas URL, iteras a través de cada una de ellas. Envías otra solicitud HTTP para descargar los datos de la imagen asociados con cada URL.

> *Es importante señalar que esta operación se realiza de manera sincrónica en tu implementación actual. Eso significa que cada imagen se descarga una a la vez, lo que podría ser lento para páginas web con un gran número de imágenes. Para un enfoque más eficiente, se podrían explorar métodos de programación asíncrona o la biblioteca concurrent.futures para descargar múltiples imágenes simultáneamente.*

```markdown
# Find all img elements
img_elements = soup.find_all('img')

# Iterate over each img elements
for img_element in img_elements:
...
```

Completa el código a continuación para que funcione:

Crea un nuevo archivo python y llámalo `automate_url_captioner.py`, y copia el código a continuación. Completa la parte en blanco para que funcione.

The content of this lab is licensed under Apache 2.0

python

```python
import requests
from PIL import Image
from io import BytesIO
from bs4 import BeautifulSoup
from transformers import AutoProcessor, BlipForConditionalGeneration

# Load the pretrained processor and model
processor = # fill the pretrained model
model = # load the blip model

# URL of the page to scrape
url = "https://en.wikipedia.org/wiki/IBM"

headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get(url, headers=headers)
print("Status:", response.status_code, "HTML size:", len(response.text))

soup = BeautifulSoup(response.text, "html.parser")
img_elements = soup.find_all("img")
print(f"Found {len(img_elements)} <img> tags")

with open("captions.txt", "w", encoding="utf-8") as caption_file:
    for idx, img_element in enumerate(img_elements, start=1):
        # Try different attributes
        img_url = img_element.get("src") or img_element.get("data-src")
        if not img_url and img_element.has_attr("srcset"):
            img_url = img_element["srcset"].split()[0]
        if not img_url:
            continue

        # Skip SVGs directly
        if img_url.endswith(".svg") or ".svg" in img_url:
            continue

        # Fix relative URLs
        if img_url.startswith("//"):
            img_url = "https:" + img_url
        elif img_url.startswith("/"):
            img_url = "https://en.wikipedia.org" + img_url
        elif not img_url.startswith("http"):
            continue

        try:
            r = requests.get(img_url, timeout=10, headers=headers)
            raw_image = Image.open(BytesIO(r.content))

            # Skip very small images
            if raw_image.size[0] * raw_image.size[1] < 200:
```

The content of this lab is licensed under Apache 2.0

```
            raw_image = raw_image.convert("RGB")

            # Process the image with a text prompt
            text = "the image of"
            inputs = processor(images=raw_image, text=text, return_tensors
            out = model.generate(**inputs, max_new_tokens=50)
            caption = processor.decode(out[0], skip_special_tokens=True)

            caption_file.write(f"{img_url}: {caption}\n")
            print(f"[{idx}] Caption saved")

        except OSError:
            # Skip images PIL cannot open (SVG, ICO, corrupt files)
            continue
        except Exception as e:
            print(f"[{idx}] Error: {e}")
            continue
```

▶ Haz clic aquí para la respuesta

Como resultado, tendrás un nuevo archivo en el explorador (en el mismo directorio) con el nombre `captions.txt` (como se muestra en la imagen a continuación).



Captions.txt

Bonificación: Generación de subtítulos para archivos locales (Ejecutar localmente si usas Blip2)

Con algunas modificaciones, puedes adaptar el código para operar en imágenes

The content of this lab is licensed under Apache 2.0

imagen en un directorio específico y luego escribir los subtítulos generados en un archivo de texto.

Además, puedes hacer uso del modelo Blip2, que es un modelo preentrenado más potente para la generación de subtítulos de imágenes. De hecho, puedes incorporar fácilmente cualquier nuevo modelo preentrenado que esté disponible, ya que se desarrollan continuamente para ser más potentes. En el ejemplo a continuación, demostramos el uso del modelo Blip2. Sin embargo, ten en cuenta que el modelo Blip2 requiere 10 GB de espacio, por lo que no se puede ejecutar en el entorno CloudIDE.

python

```
import glob
...
# Specify the directory where your images are
image_dir = "/path/to/your/images"
image_exts = ["jpg", "jpeg", "png"]  # specify the image file extensions t

# Open a file to write the captions
with open("captions.txt", "w") as caption_file:
# Iterate over each image file in the directory
    for image_ext in image_exts:
        for img_path in glob.glob(os.path.join(image_dir, f"*.{image_ext}"
            # Load your image
            raw_image = Image.open(img_path).convert('RGB')
            ...
```

Intenta implementarte a ti mismo.

▶ Haz clic aquí para ver la versión completa

The content of this lab is licensed under Apache 2.0

Conclusión

¡Felicidades por completar este proyecto guiado! Ahora has dominado la inteligencia artificial para la generación de subtítulos de imágenes utilizando Gradio.

Próximos pasos

Puedes desplegar tu aplicación en internet utilizando IBM Code Engine. En la siguiente sección opcional, ofrecemos una guía paso a paso para ayudarte a hacerlo.

Al final de este proyecto guiado, desplegarás una aplicación en un clúster de Kubernetes utilizando IBM Code Engine. Usaste un clúster compartido proporcionado por la IBM Developer Skills Network. Si deseas desplegar tu aplicación en contenedor y obtener una URL permanente de la aplicación fuera de la CLI de Code Engine en tu máquina local, puedes aprender más sobre Kubernetes y contenedores. Puedes obtener tu propio clúster de Kubernetes gratuito y tu propio IBM Container Registry gratuito.

Author(s)

Sina Nazeri

© IBM Corporation. Todos los derechos reservados.

Asigna Nombres Significativos a

The content of this lab is licensed under Apache 2.0

Tus Fotos con IA

Introducción

Las imágenes, ricas en información no aprovechada, a menudo quedan fuera del radar de los motores de búsqueda y sistemas de datos. Transformar estos datos visuales en un lenguaje legible por máquinas no es una tarea fácil, pero es aquí donde la IA de subtitulación de imágenes resulta útil. Aquí te mostramos cómo la IA de subtitulación de imágenes puede marcar la diferencia:

- Mejora la accesibilidad: Ayuda a las personas con discapacidad visual a comprender el contenido visual.
- Mejora el SEO: Asiste a los motores de búsqueda en la identificación del contenido de las imágenes.
- Facilita el descubrimiento de contenido: Permite un análisis y categorización eficientes de grandes bases de datos de imágenes.
- Apoya las redes sociales y la publicidad: Automatiza la generación de descripciones atractivas para contenido visual.
- Aumenta la seguridad: Proporciona descripciones en tiempo real de las actividades en grabaciones de video.
- Ayuda en la educación y la investigación: Asiste en la comprensión e interpretación de materiales visuales.
- Ofrece soporte multilingüe: Genera subtítulos de imágenes en varios idiomas para audiencias internacionales.
- Permite la organización de datos: Ayuda a gestionar y categorizar grandes conjuntos de datos visuales.
- Ahorra tiempo: La subtitulación automatizada es más eficiente que los esfuerzos manuales.
- Aumenta la participación del usuario: Los subtítulos detallados pueden hacer que el contenido visual sea más atractivo e informativo.

The content of this lab is licensed under Apache 2.0

Objetivos de aprendizaje

Al final de este proyecto, podrás:

- Implementar una herramienta de subtitulación de imágenes utilizando el modelo BLIP de los Transformers de Hugging Face.
- Usar Gradio para proporcionar una interfaz amigable para tu aplicación de subtitulación de imágenes.
- Adaptar la herramienta a escenarios comerciales del mundo real, demostrando sus aplicaciones prácticas.



Generado por IA y mejorado por humanos

Configuración del entorno… →

The content of this lab is licensed under Apache 2.0